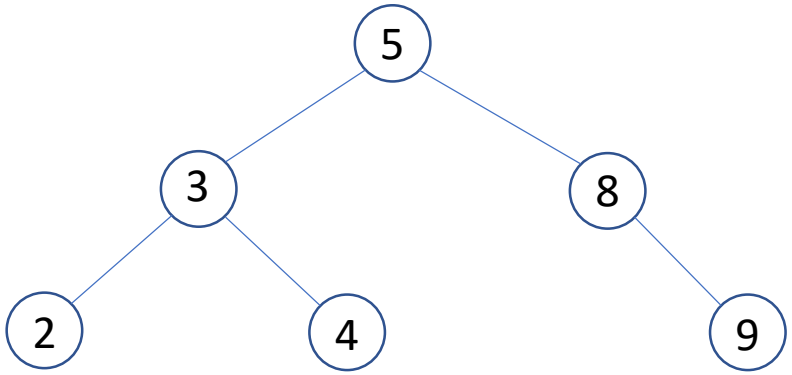
Árvores Binárias de Pesquisa com Balanceamento

Prof. Henrique Batista da Silva

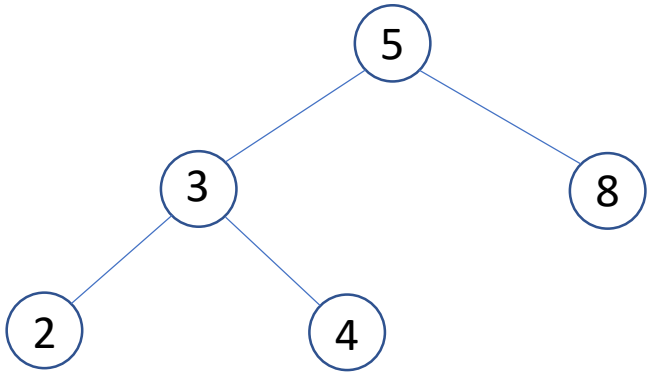
Árvores Binárias de Pesquisa com Balanceamento

- Árvore completamente balanceada:
  - Árvores cujo nível das folhas difere no máximo de 1 (ou uma árvore binária é balanceada se, em cada um de seus nós, a altura das subárvores esquerda e direita diferem em no máximo 1 unidade). Uma árvore binária balanceada com n nós tem altura próxima de $\log n$
  - Árvore completamente balanceada:



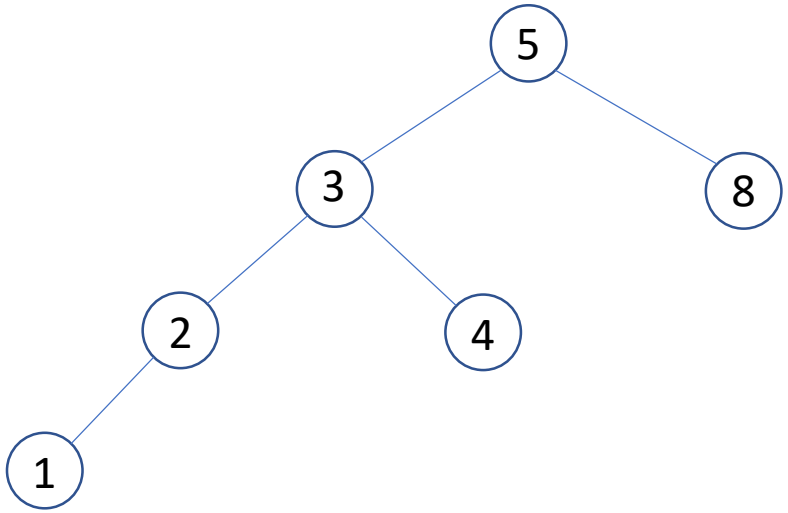
Árvores Binárias de Pesquisa com Balanceamento

- Árvore completamente balanceada:
  - Árvores cujo nível das folhas difere no máximo de 1 (ou uma árvore binária é balanceada se, em cada um de seus nós, a altura das subárvores esquerda e direita diferem em no máximo 1 unidade). Uma árvore binária balanceada com n nós tem altura próxima de $\log n$
  - Árvore completamente balanceada:



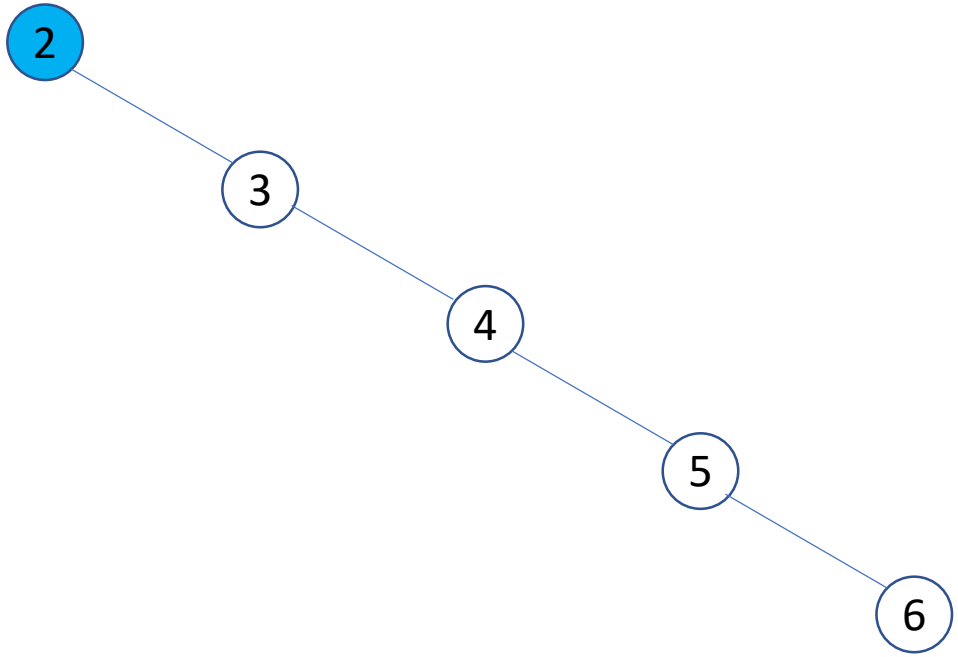
Árvores Binárias de Pesquisa com Balanceamento

- Árvore completamente balanceada:
  - Árvores cujo nível das folhas difere no máximo de 1 (ou uma árvore binária é balanceada se, em cada um de seus nós, a altura das subárvores esquerda e direita diferem em no máximo 1 unidade). Uma árvore binária balanceada com n nós tem altura próxima de $\log n$
  - Árvore **não** balanceada:



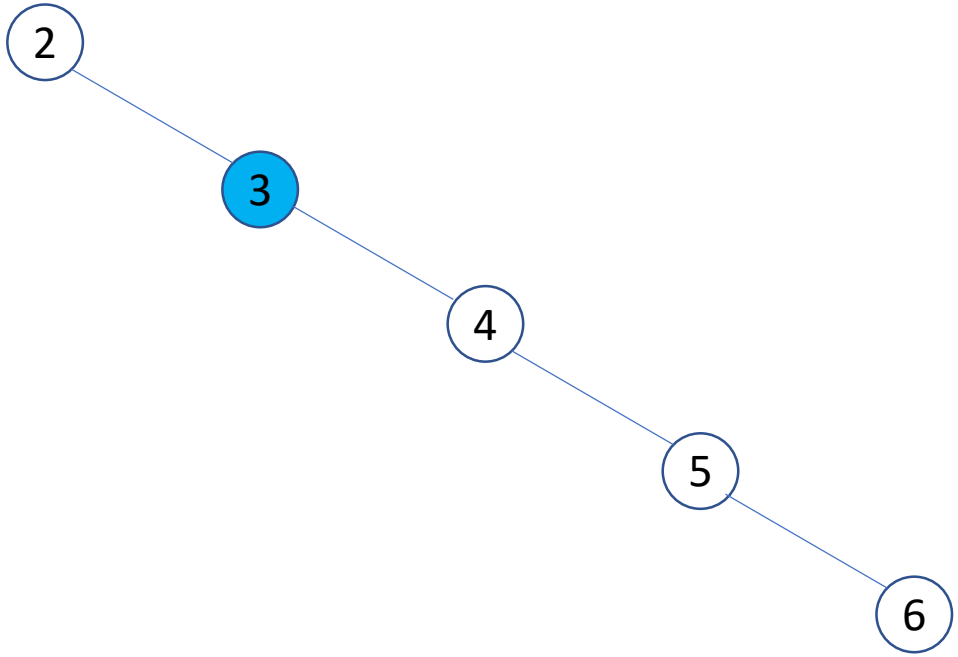
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore não balanceada:



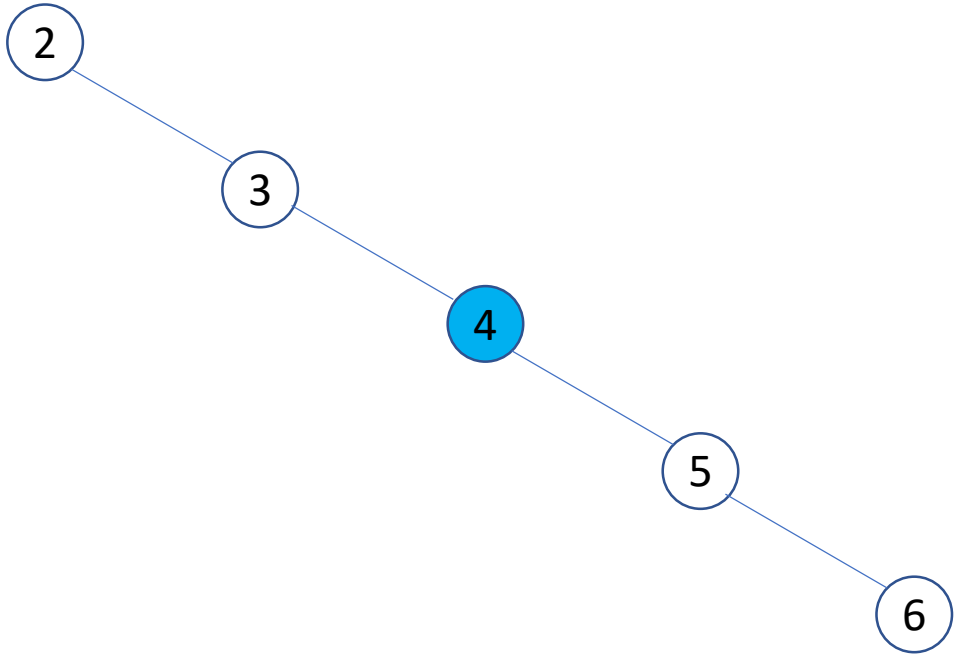
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore não balanceada:



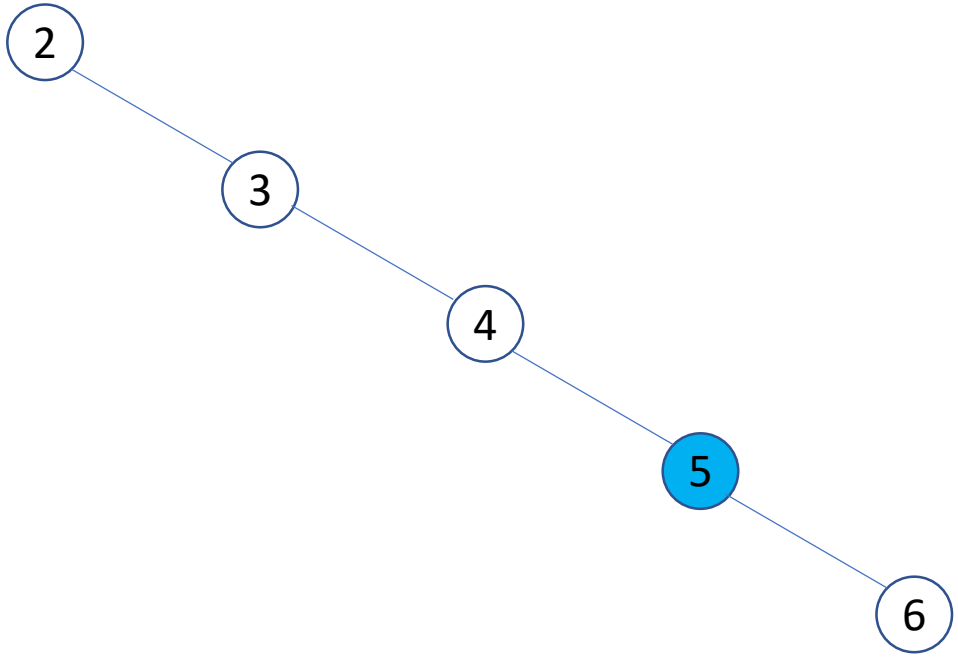
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore não balanceada:



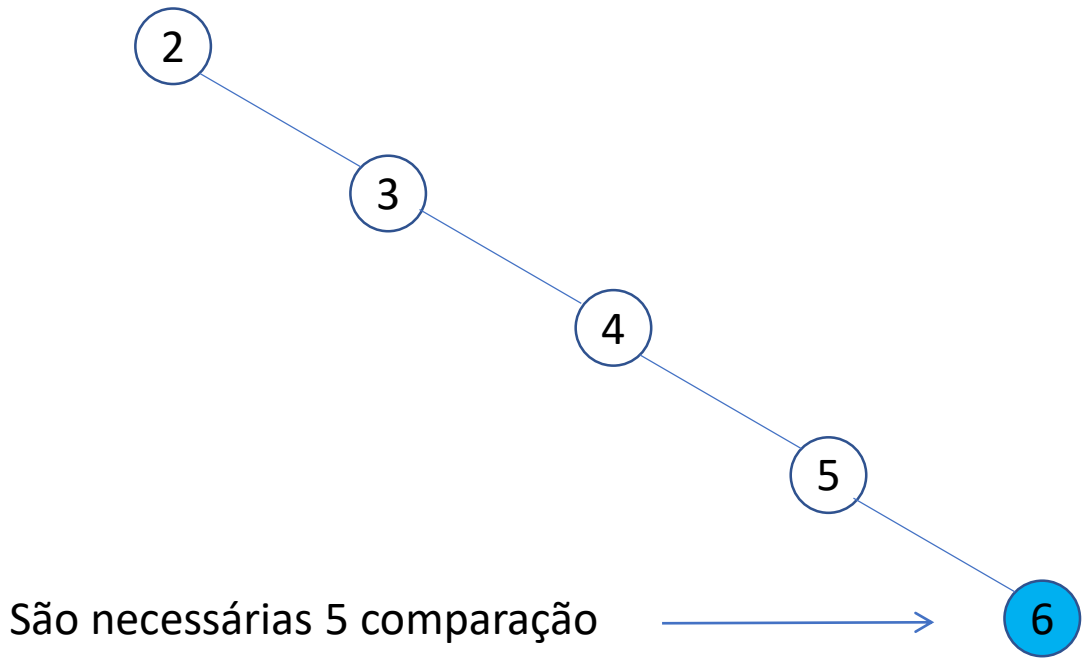
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore não balanceada:



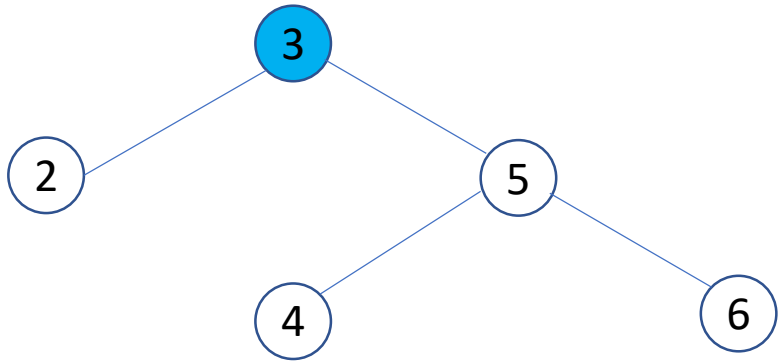
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore não balanceada:



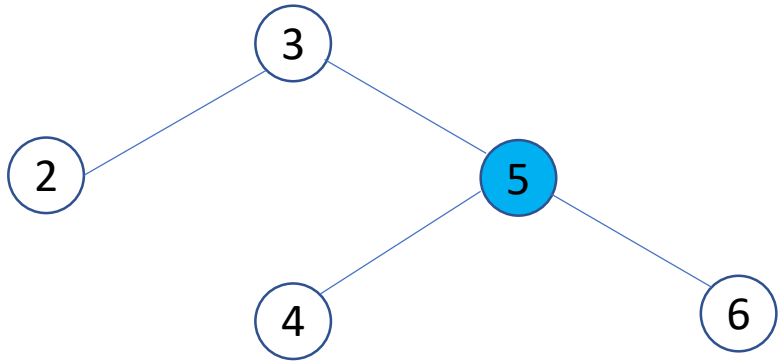
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore balanceada:



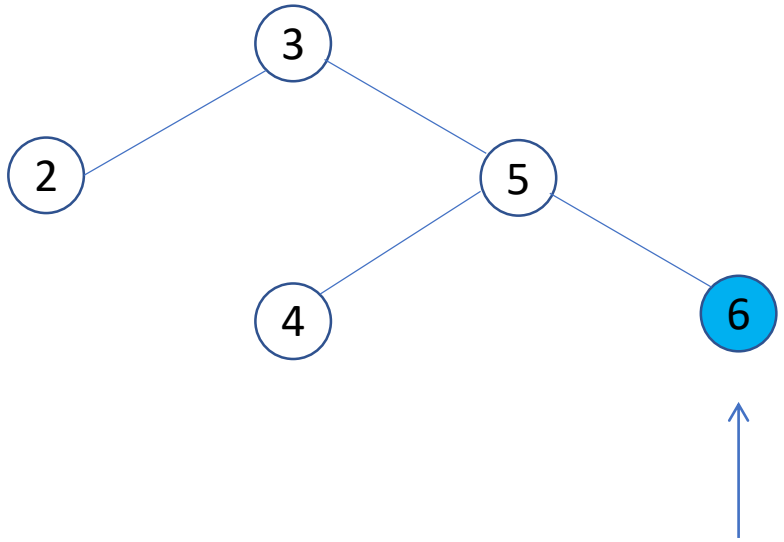
Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore balanceada:



Árvores Binárias de Pesquisa com Balanceamento

- Encontrar o elemento 6 na árvore balanceada:



São necessárias 3 comparação

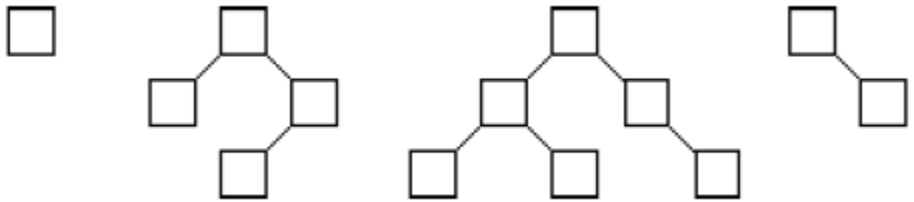
Árvores Binárias de Pesquisa com Balanceamento

- Minimiza tempo médio de pesquisa para uma distribuição uniforme das chaves:
  - cada chave é igualmente provável de ser usada em uma pesquisa.
- Contudo, custo para manter a árvore completamente balanceada após cada inserção é muito alto.

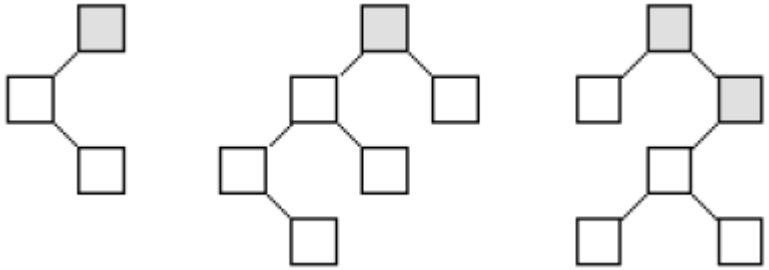
Árvore AVL

- Adelson-Velskii e Landis (1962)
- Heurística : a altura das sub árvores esquerda e direita de qualquer nó pode ser diferente em apenas uma unidade.

- Árvores AVL:



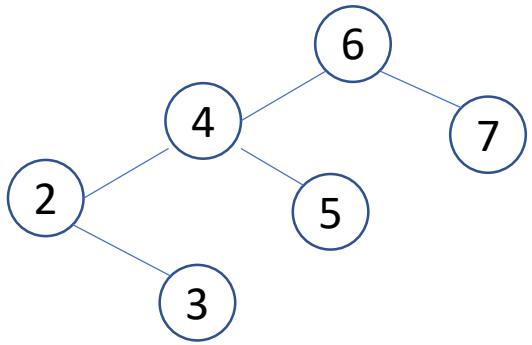
- Árvores não AVL:



Rebalanceamento

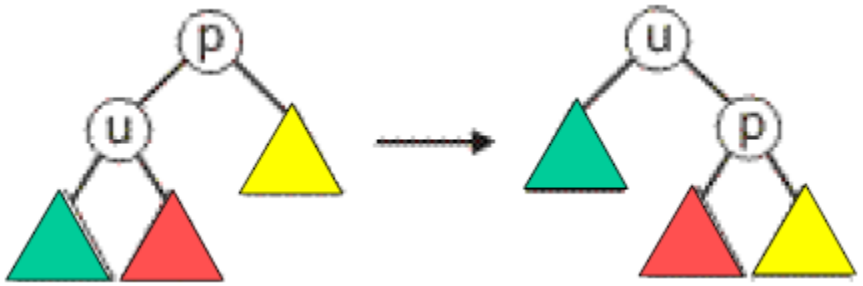
- Inserção na AVL:
  - Apenas os nós que estão no caminho entre a raiz e o nó recém inserido são afetados.

- Caso 1: A altura da sub árvore de raiz 4 difere de mais de 1 unidade da de chave 7.



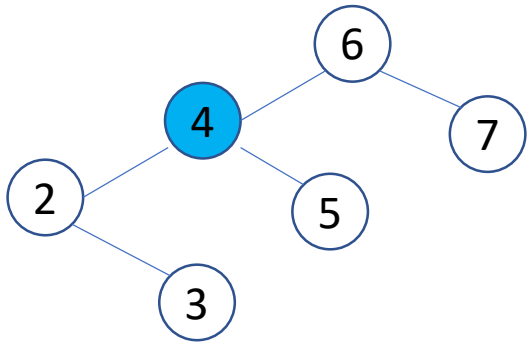
Rebalanceamento

- Caso 1.1: Lado esquerdo desbalanceado:
  - Rotação simples para a direita.



Rebalanceamento

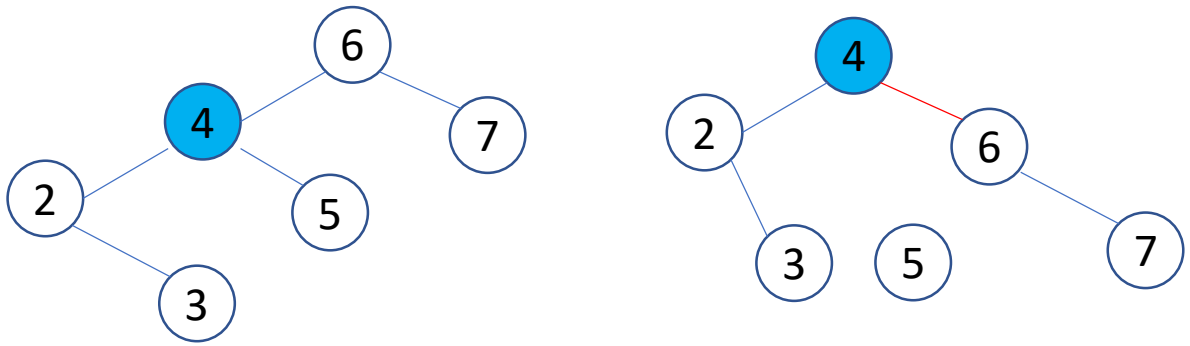
- Caso 1.1: Lado esquerdo desbalanceado:
  - Rotação simples para a direita.



1: Seja Y (4) o filho à esquerda de X (6)

Rebalanceamento

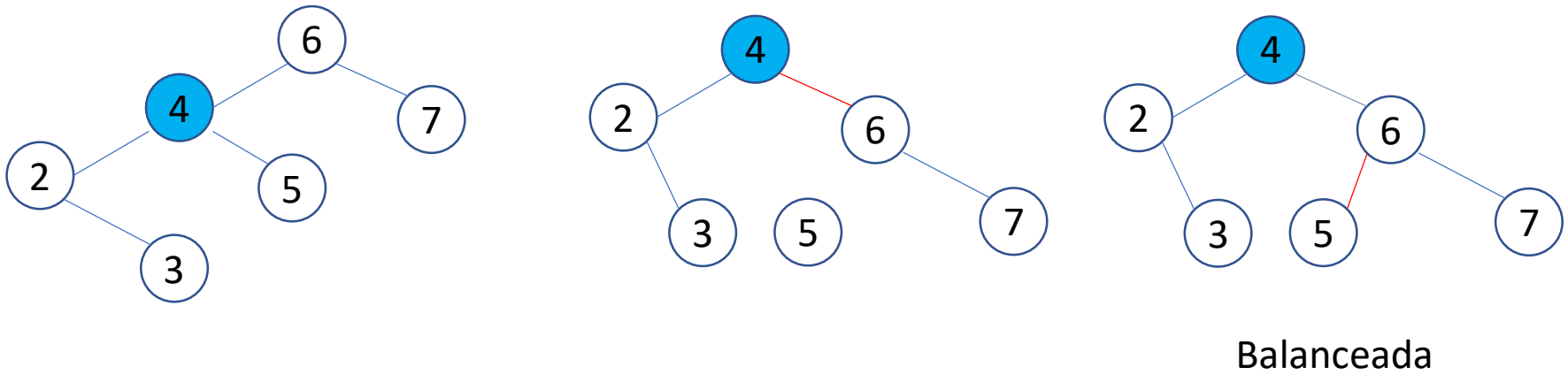
- Caso 1.1: Lado esquerdo desbalanceado:
  - Rotação simples para a direita.



1: Seja Y (4) o filho à esquerda de X (6)
2: Torne X (6) o filho à direita de Y (4)

Rebalanceamento

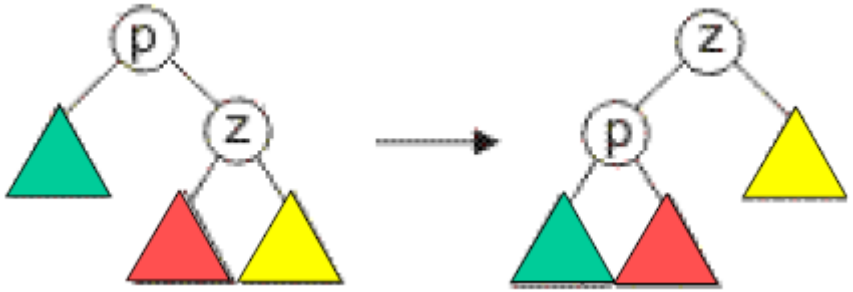
- Caso 1.1: Lado esquerdo desbalanceado:
  - Rotação simples para a direita.



1: Seja Y (4) o filho à esquerda de X (6)
2: Torne X (6) o filho à direita de Y (4)
3: Torne o filho à direita de Y (4) o filho à esquerda de X (6).

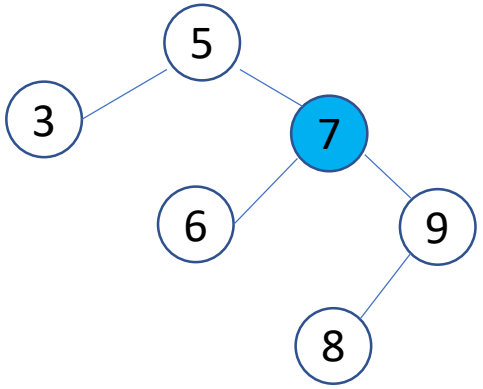
Rebalanceamento

- Caso 1.2: Lado direito desbalanceado:
  - Rotação simples para a esquerda.



Rebalanceamento

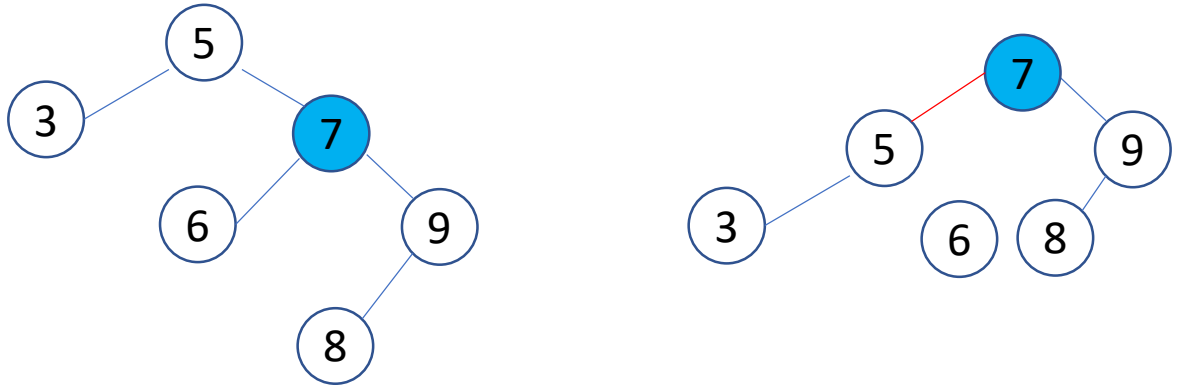
- Caso 1.2: Lado direito desbalanceado:
  - Rotação simples para a esquerda.



1: Seja Y (7) o filho à direita de X (5)

Rebalanceamento

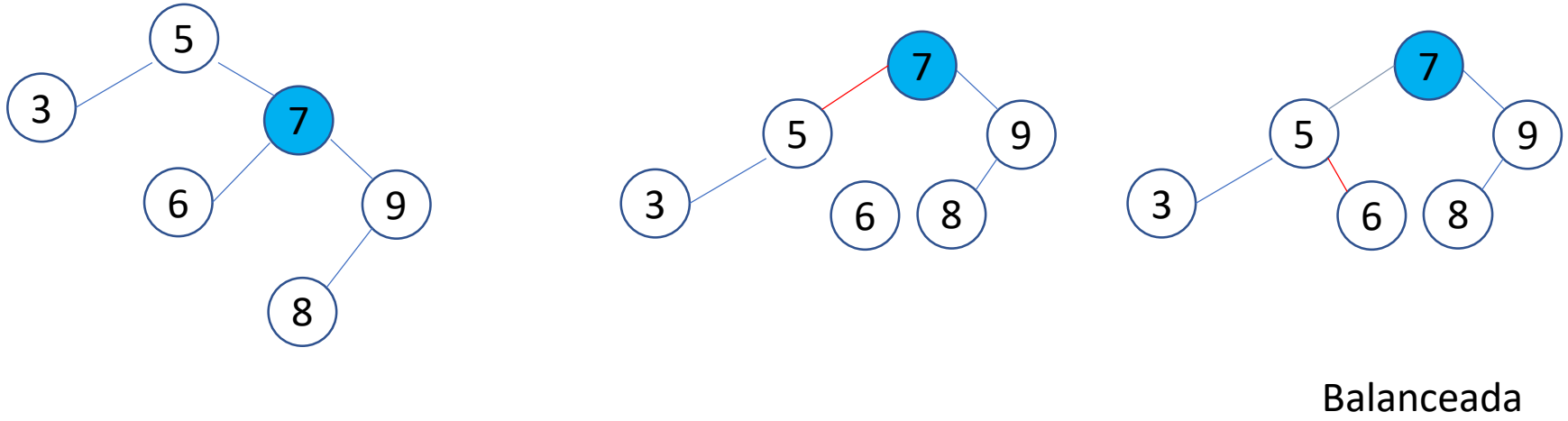
- Caso 1.2: Lado direito desbalanceado:
  - Rotação simples para a esquerda.



1: Seja Y (7) o filho à direita de X (5)
2: Torne X (5) o filho à esquerda de Y (7)

Rebalanceamento

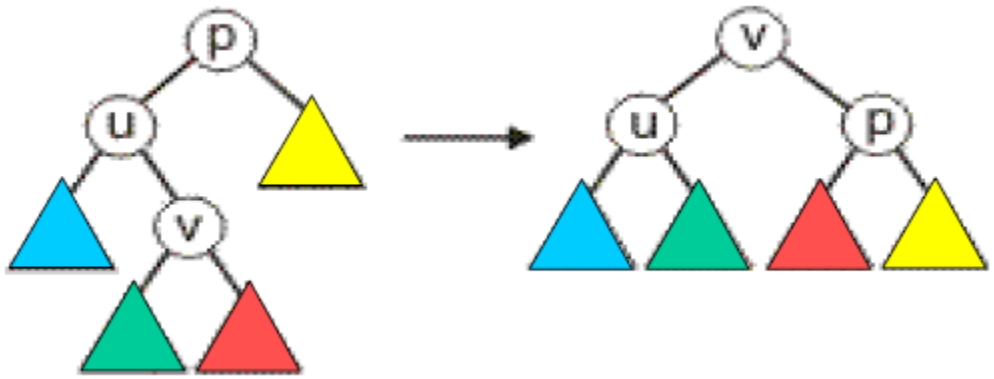
- Caso 1.2: Lado direito desbalanceado:
  - Rotação simples para a esquerda.



1: Seja Y (7) o filho à direita de X (5)
2: Torne X (5) o filho à esquerda de Y (7)
3: Torne o filho à esquerda de Y (7) o filho à direita de X (5).

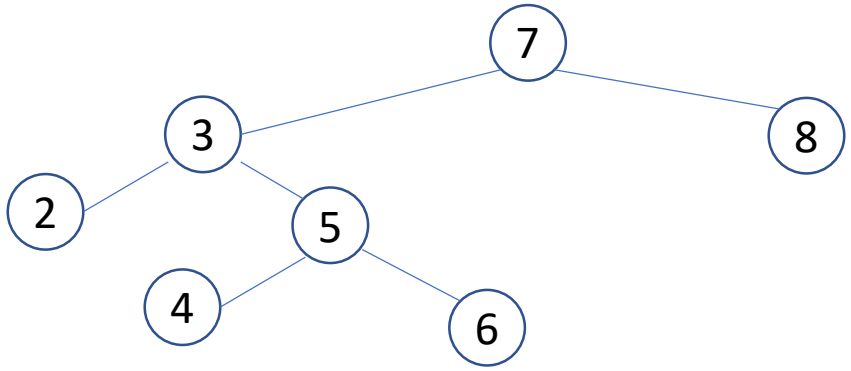
Rebalanceamento

- Caso 2: Lado esquerdo desbalanceado. Se um nó estiver desbalanceado e seu filho estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a direita.



Rebalanceamento

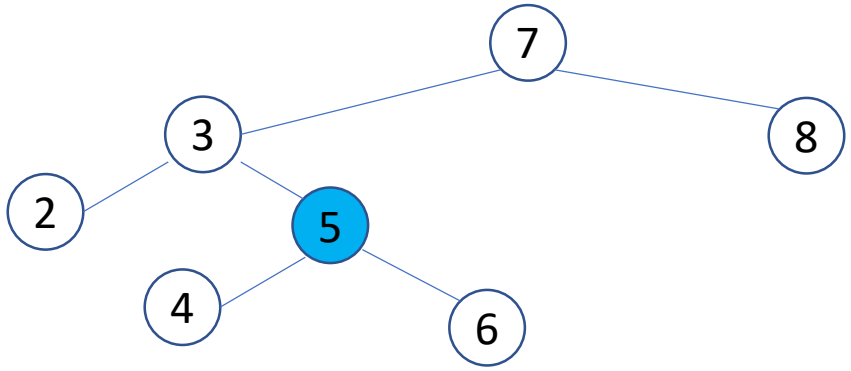
- Caso 2.1: Lado esquerdo desbalanceado. Se um nó estiver desbalanceado (nó 3) e seu filho (nó 5) estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a direita.



Rotação simples à esquerda, e
Rotação simples à Direita.

Rebalanceamento

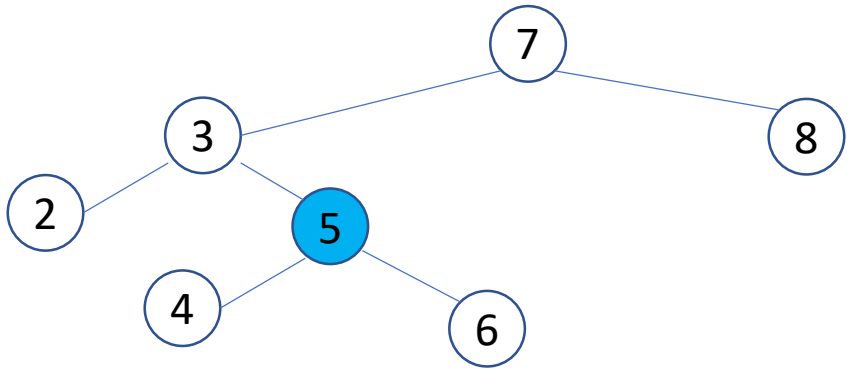
- Caso 2.1: Lado esquerdo desbalanceado. Se um nó estiver desbalanceado (nó 3) e seu filho (nó 5) estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a direita.

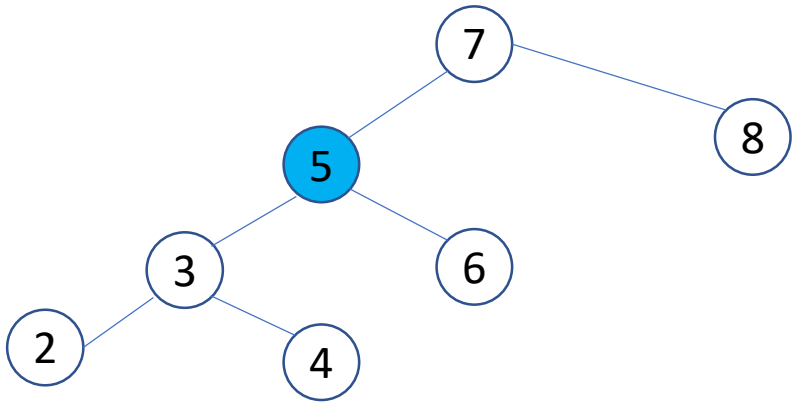


Rotação simples à esquerda, e
Rotação simples à Direita.

Rebalanceamento

- Caso 2.1: Lado esquerdo desbalanceado. Se um nó estiver desbalanceado (nó 3) e seu filho (nó 5) estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a direita.



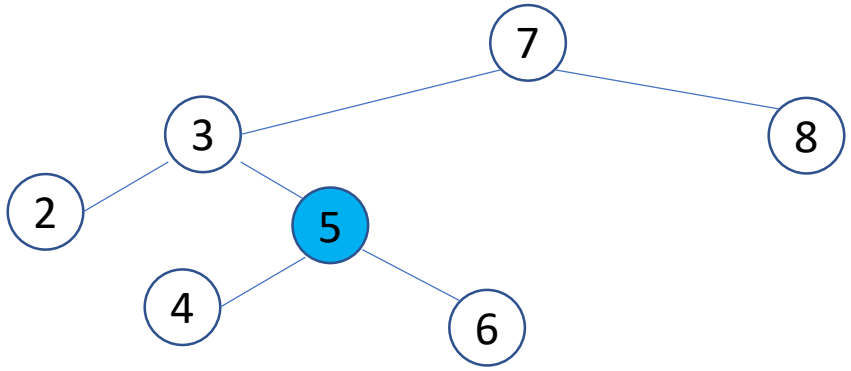


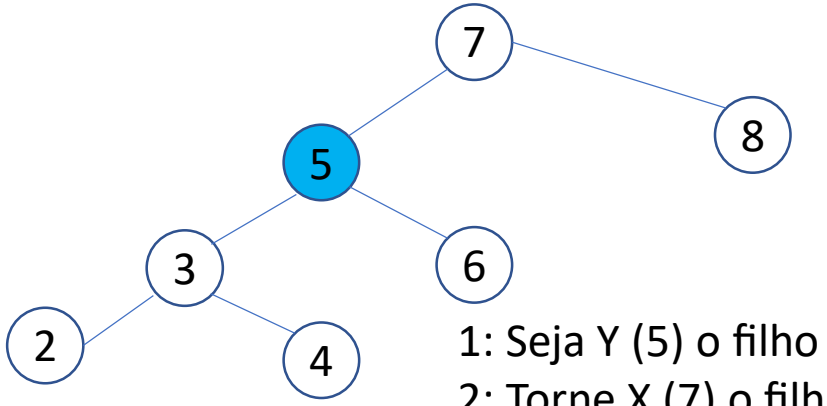
Rotação simples à esquerda, e
Rotação simples à Direita.

1: Seja Y (5) o filho à direita de X (3)
2: Torne X (3) o filho à esquerda de Y (5)
3: Torne o filho à esquerda de Y (5) o filho à direita de X (3).

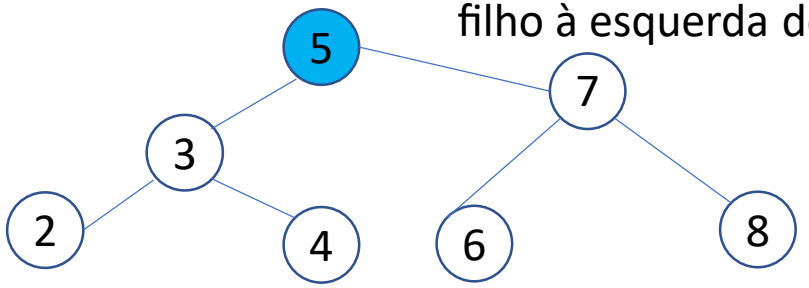
Rebalanceamento

- Caso 2.1: Lado esquerdo desbalanceado. Se um nó estiver desbalanceado (nó 3) e seu filho (nó 5) estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a direita.





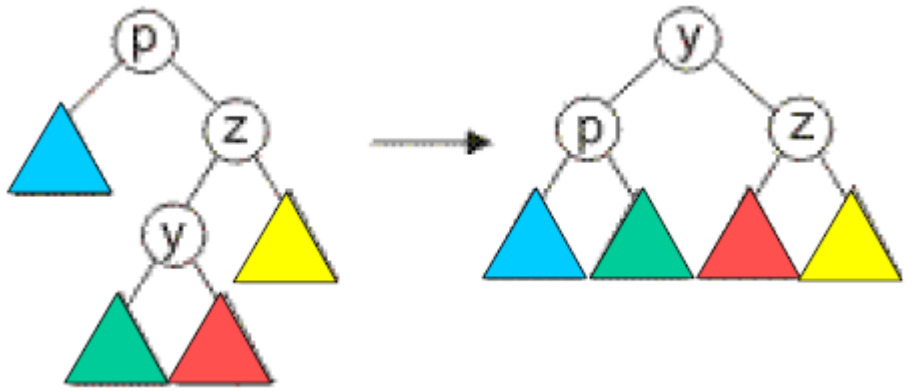
1: Seja Y (5) o filho à esquerda de X (7)
2: Torne X (7) o filho à direita de Y (5)
3: Torne o filho à direita de Y (5) o filho à esquerda de X (7).



Rotação simples à esquerda, e
Rotação simples à Direita.

Rebalanceamento

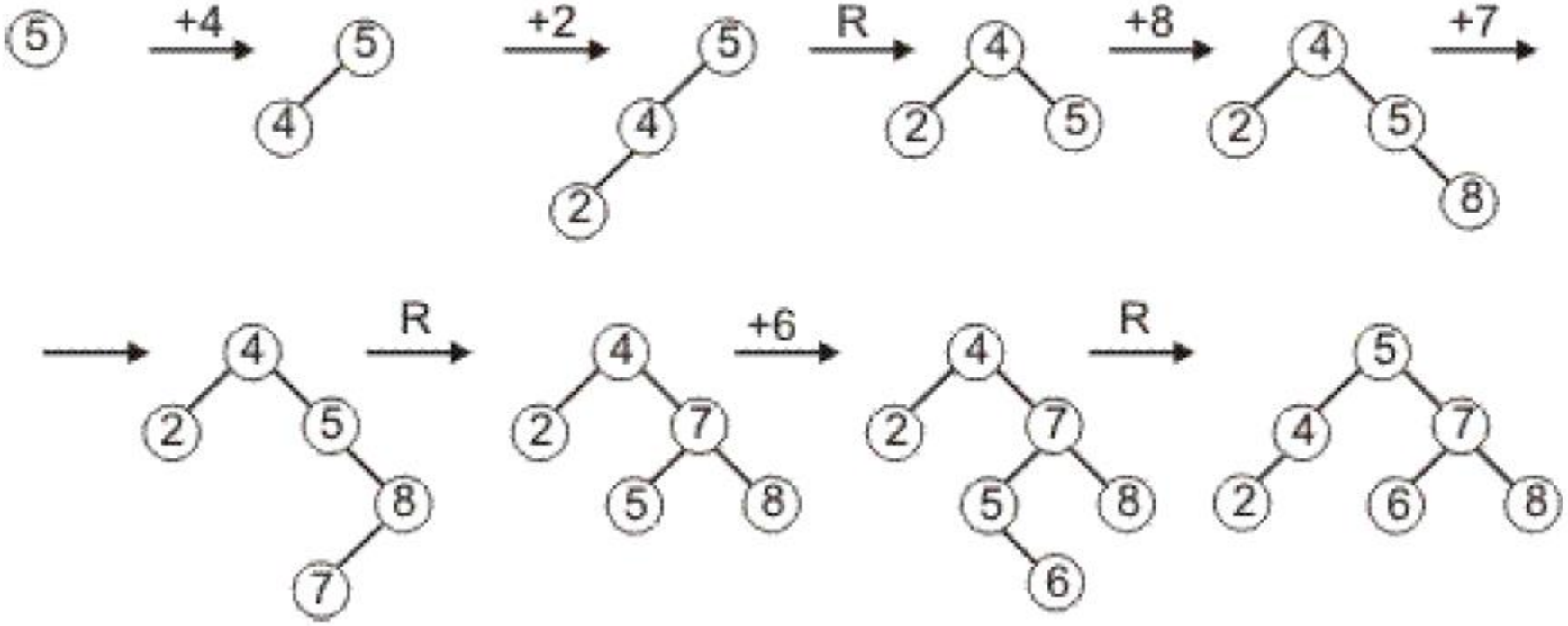
- Caso 2.2: Lado direito desbalanceado. Se um nó estiver desbalanceado e seu filho estiver inclinado no sentido inverso ao pai:
  - Rotação dupla para a esquerda.



Rotação simples à direita, e
Rotação simples à esquerda.

Rebalanceamento

- Exemplo de Inserções em Árvore AVL:
  - Árvore começa apenas com o nó com chave 5 e, em seguida, foram acrescentados os nós com chaves: 4, 2, 8, 7 e 6, nesta ordem:

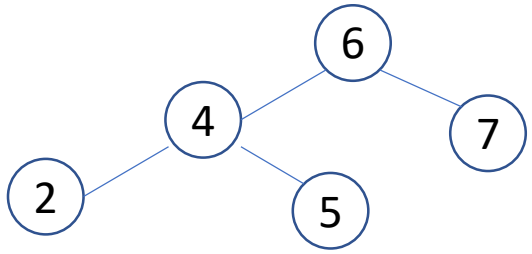


Rebalanceamento

- Remoção:
  - Remover um nó de uma árvore AVL consiste em remover diretamente o nó caso ele seja uma folha e verificar o balanceamento realizando as rotações se necessárias.
  - Caso não seja um nó folha, troca-se com o nó mais à direita do filho esquerdo.

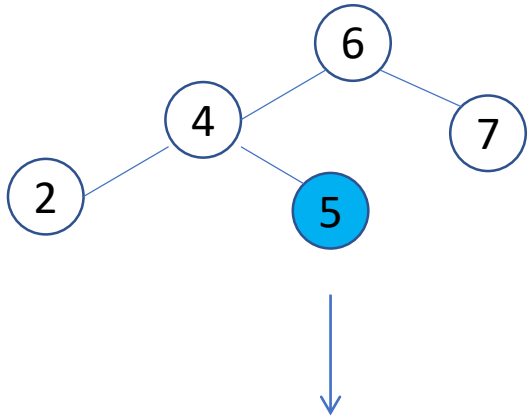
Rebalanceamento

- Remover: 5



Rebalanceamento

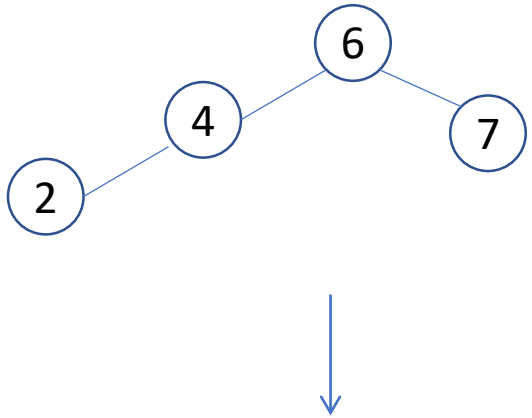
- Remover: 5



Nó folha, remover e verificar balanceamento

Rebalanceamento

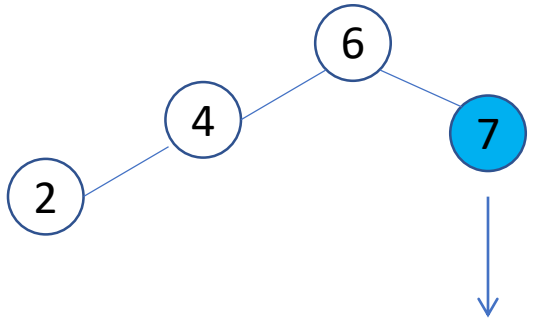
- Remover: 5



Árvore ainda balanceada

Rebalanceamento

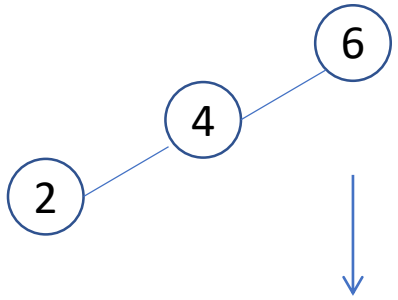
- Remover: 7



Nó folha, remover e verificar balanceamento

Rebalanceamento

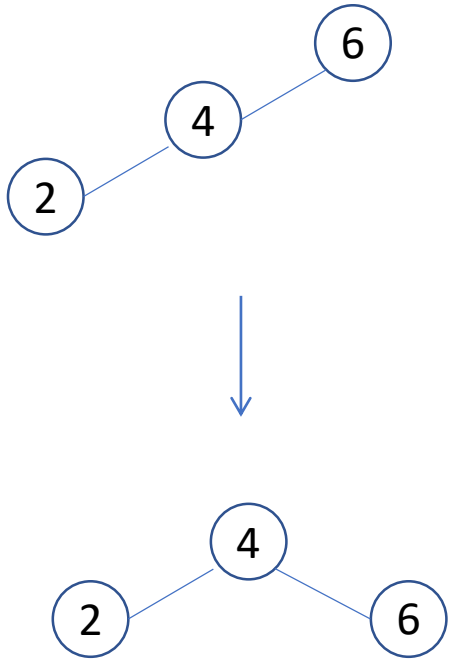
- Remover: 7



Árvore desbalanceada, realizar rotação à direita.

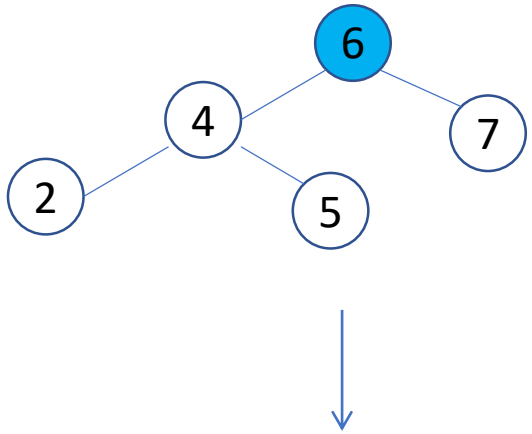
Rebalanceamento

- Remover: 7



Rebalanceamento

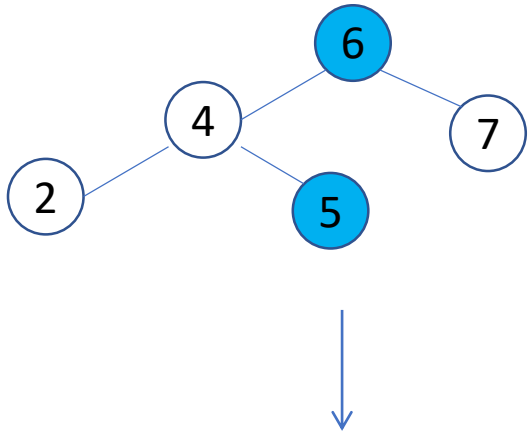
- Remover: 6



Nó não folha, trocar com o nó mais à direita do filho esquerdo.

Rebalanceamento

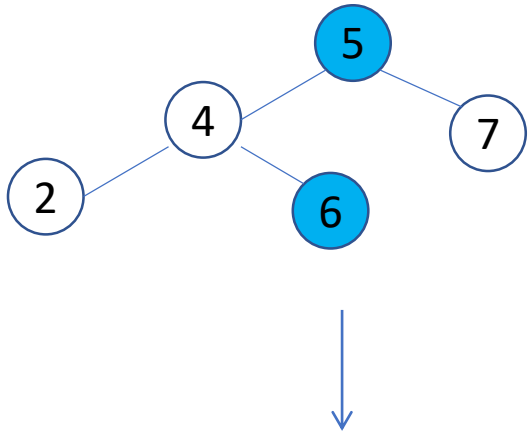
- Remover: 6



Nó não folha, trocar com o nó mais à direita do filho esquerdo.

Rebalanceamento

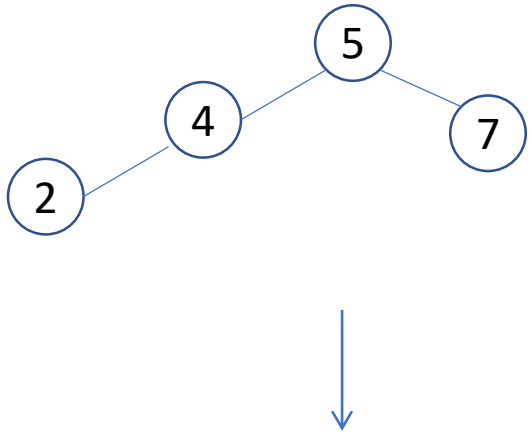
- Remover: 6



Exclui o nó, que agora é nó folha. E verificar o balanceamento da árvore.

Rebalanceamento

- Remover: 6



Árvore Balanceada

Exercícios

- Represente a árvore binária obtida a partir da inserção de elementos com as seguintes chaves: 5 / 6 / 9 / 2 / 1 / 4 / 7 / 10.
- Para a árvore da questão anterior, represente a árvore após a remoção dos elementos: 10 e 5 (nesta ordem).
- Represente a árvore AVL obtida a partir da inserção dos mesmos elementos da questão anterior (5 / 6 / 9 / 2 / 1 / 4 / 7 / 10)
- Para a árvore AVL da questão anterior, represente a árvore após a remoção dos elementos com as chaves: 1 e 4 (nesta ordem).

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).